

Placement Portal Application V2 – Final Project Report

1. Student Details

Name: Aratrika Chakrabarti

Roll Number: 23f2000404

Email: 23f2000404@ds.study.iitm.ac.in

About Me: I am a BS in Data Science student at IIT Madras with an interest in web application development and practical software systems. I enjoy building applications, testing complete workflows, and understanding how the different parts of a web application work together.

2. Project Details

- **Project Title: Placement Portal Application (PPA) — V2**
- **Problem Statement:** To design and build a web-based placement portal that connects students, companies, and administrators. The system manages company approval, placement drives, student applications, interviews, feedback, and final placement records.
- **Approach:** The application was developed as a role-based placement portal with separate workflows for Admin, Company, and Student users. Flask and SQLAlchemy are used for the backend, while Vue.js components provide the frontend interface and communicate with the backend through REST APIs. The frontend uses Vue.js 3 through CDN with Vue Router, avoiding a separate frontend build system and keeping the project simple to run locally.

The system was built incrementally, starting with authentication, database models, and core placement workflows, followed by resume handling, application tracking, interview scheduling, and placement management. Celery with Redis was added for background jobs such as reminders, reports, and CSV exports, while Redis caching was used for frequently accessed data. The final implementation was tested using seeded/demo data covering the main student, company, and admin workflows.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) during development to assist with code suggestions, debugging, documentation, sample data, and reviewing parts of the implementation. The extent of AI/LLM usage was significant, but the final integration, testing, project decisions, and end-to-end verification were done manually.

The extent of AI usage was approximately 25–30%.

4. Technologies and Frameworks Used

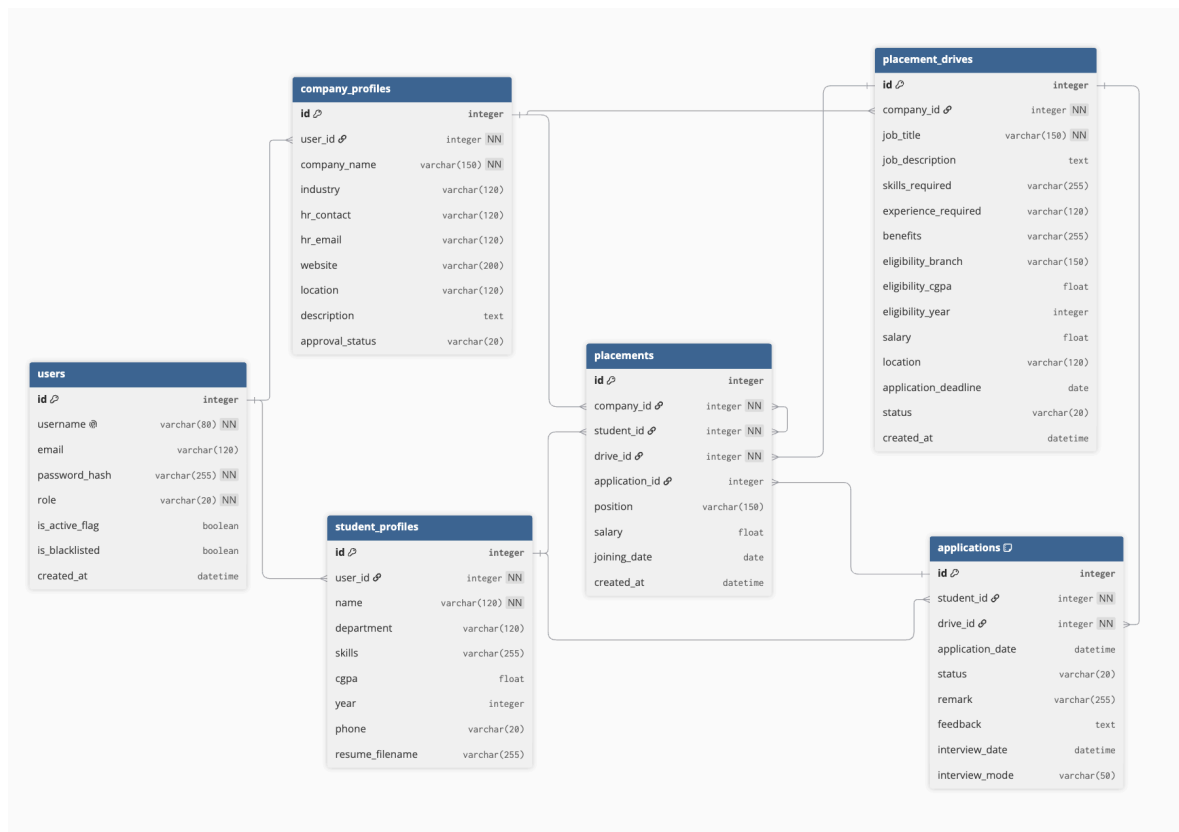
Component	Technology	Purpose
Backend	Flask 3.0.3	REST API framework
Database ORM	Flask-SQLAlchemy 3.1.1 / SQLAlchemy 2.0.51	Models & DB abstraction
Authentication	Flask-Login 0.6.3	Session-based user auth
Caching	Flask-Caching 2.3.0 + Redis 5.0.7	Cache layer & broker
Async Jobs	Celery 5.4.0 + Redis	Background tasks & scheduling
Frontend	Vue.js 3.4.31 (CDN)	SPA without build step
Routing (FE)	Vue Router 4.4.0	Client-side navigation
Styling	Bootstrap 5.3.3	CSS framework (only allowed)
Database	SQLite	Lightweight local DB
Server	Werkzeug 3.0.3	Flask's WSGI server

5. Database Schema / ER Diagram

Tables:

1. User — stores login credentials, role, account status, blacklist status, and timestamps.
2. CompanyProfile — stores company details, HR contact information, location, description, and admin approval status.
3. StudentProfile — stores student details such as department, skills, CGPA, graduation year, phone number, and resume filename.
4. PlacementDrive — stores job position details, eligibility criteria, salary, deadline, company and drive status.
5. Application — links a student to a drive and stores application status, remarks, feedback, and interview details.
6. Placement — stores the final placement record including company, position, salary, joining date, and related application.

Relationships: User has one company or student profile. Companies have many placement drives and placements. Students have many applications and placements. A drive has many applications. A unique constraint on student_id + drive_id prevents duplicate applications.



Above is the project's db schema made by using <https://dbdiagram.io>

6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/register	POST	Register a student or company account
/api/auth/login	POST	Authenticate user and create session
/api/admin/companies	GET	List/search company registrations for admin review
/api/company/drives	POST	Create a placement drive

/api/student/drives	GET	List approved drives with search/filter options
/api/student/drives/{id}/apply	POST	Apply to an eligible placement drive
/api/company/applications/{id}/status	POST	Update application status and placement details
/api/company/applications/{id}/schedule-interview	POST	Schedule an interview
/api/student/export-csv	POST	Start an asynchronous student CSV export
/api/company/export-csv	POST	Start an asynchronous company CSV export

7. Architecture and Features (optional)

Architecture Overview:

- app.py – Flask application entry point and application factory
- /models.py – SQLAlchemy database models
- /routes – Flask Blueprints for authentication, admin, company and student operations
- /templates – single Jinja2 entry point for the Vue application
- /static – Vue components, Vue Router, API wrapper, CSS and Bootstrap-based UI

Implemented Features:

- User registration and login
- Role-based access for admin, company and student
- Company approval and placement drive approval
- Student drive search, eligibility checking and applications
- Resume upload and validation
- Application status funnel, feedback and interview scheduling
- Placement records and offer/placement confirmation
- CSV export with asynchronous Celery jobs

Additional Features:

- Celery deadline and interview reminders
 - Admin-wide and per-company monthly reports
 - Redis caching for selected dashboard and search APIs
 - Cache invalidation after write operations
 - Async job-status polling for CSV exports
 - Duplicate-application protection and server-side eligibility validation
 - Application deadline validation during drive creation and application
 - User-specific authorization for generated CSV exports
 - IDOR (Insecure Direct Object Reference) prevention
-

8. Video Presentation

Drive Link: <https://drive.google.com/file/d/110E2ORbtE48yTJGjJwTvispl3zSPpPGh/view?usp=sharing>
